写点排序算法的笔记：

插入排序 InsertionSort O(n^2)

```
void insertSort(int arr[], int begin, int end)
{
    // int length = end - begin + 1;
    for (int i = begin + 1; i < end + 1; i++)
    {
        for (int j = i; j > begin; j--)
        {
            if (arr[j] < arr[j - 1])
            {
                swap(&arr[j], &arr[j - 1]);
            }else
            {
                break;
            }
        }
    }
}
```

插入排序的本质就是排个头，先把第一个固定住，然后看第二个，第二个要是比第一个矮，就swap一下。

然后看第三个，第三个有没有第二个矮，要是矮，swap，此时第三个变成了第二个，再判断一下它和第一个的关系……

循环往复，最后就排序完毕了。

快速排序 QuickSort O(nlogn)

```
void QuickSort(int array[], int low , int high)
{
    if(low<high){
    int index = array[low];
    int i = low;
    int j = high;
    while (i<j)
    {
        while (i<j && array[j] >= index)
        {
            j--;
        }

        if (i<j)
        {
```

```
            swap(&array[i], &array[j]);
            i++;
        }

        while (i<j && array[i]<index)
        {
            i++;
        }

        if(i<j)
        {
            swap(&array[i], &array[j]);
            j--;
        }
    }
    array[i] = index;
    QuickSort(array,low,i-1);
    QuickSort(array,i+1,high);
    }
}
```

快速排序是我接触的第一个排序算法，也是第一个利用到了分治和递归思想的算法。

说大白话就是**分堆**，我们先选一个“标准/标杆”(index), 这里我直接选的是第一项，然后我们把所有小于index的元素，都swap到index左边，大于index的元素都swap到右边。这样我们就获得了两堆数。之后对着这两堆数进行递归操作，反复的分堆分堆，分到什么时候呢？分到每个堆只有一个数字了，这也就意味着我们分完了。

在分堆的过程也是有讲究的，我们从数组的两端向中间移动。

```
while (i<j && array[j] >= index)
{
    j--;
}

if (i<j) // 这个判断就是防止因为i >= j导致while终止，误操作swap，下面的if也是同理
{
    swap(&array[i], &array[j]);
    i++; // 注意这个i++；
}
```

上面这段就是从最右端往左走，大的我们就留下，一旦发现小的，我们就直接swap到左边，同时从左边swap一个数字过来。可能你想问，如果我们swap过来的那个数字也小于index，应该放在左边怎么办呢？不急，我们接着往下看。

```
while (i<j && array[i]<index)
{
    i++;
```

```
}

if(i<j)
{
    swap(&array[i], &array[j]);
    j--;
}
```

这段开始我们从左端往右走，和上面的同理，小的我们留下，一旦发现大的，我们直接swap到右边，同时从右边swap一个数字过来。这个疑问又出现了，我们如果swap了一个大于index数字过来怎么办?

还好，循环会解决这些疑问。

当你进入下一轮最外层的while循环时，我们首先就会判断上一轮swap过来的**array [j]**满不满足我们的需求，如果不满足，果断扔回去。第二个也是同理。

就这样，不断的把数字扔来扔去，我们就得到了一个排好序的数组。

并归排序 mergeSort O(nlogn)

```
void merge(int arr[], int left, int mid, int right)
{
    int i, j, k;
    int l = mid - left + 1;
    int r = right - mid;

    // 创建两个临时数组 L/R
    int *L = (int *)malloc(l * sizeof(int));
    int *R = (int *)malloc(r * sizeof(int));

    // 把数字放进去
    for (i = 0; i < l; i++)
    {
        L[i] = arr[left + i];
    }
    for (j = 0; j < r; j++)
    {
        R[j] = arr[mid + 1 + j];
    }

    // 开始分吧
    i = 0;
    j = 0;
    k = left; // 众神归位
    while (i < l && j < r)
    {
        if (L[i] <= R[j])
        {
            arr[k] = L[i];
```

```
            i++;
        }
        else
        {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // 把剩下的元素处理干净
    while (i < l)
    {
        arr[k] = L[i];
        i++;
        k++;
    }

    while (j < r)
    {
        arr[k] = R[j];
        j++;
        k++;
    }

    // free一下，养成好习惯
    free(L);
    free(R);
}

void mergeSort(int arr[], int left, int right)
{
    if (left < right) // 啊啊啊啊啊啊啊啊啊啊啊啊啊啊啊啊啊，不要忘了比较这个啊啊啊啊啊啊啊啊啊啊啊啊啊啊啊啊
    {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}
```

并归排序的实质，在我理解看来就是局部的有序，然后进行整合。我们通过递归，获得了大量的size为2(或1)的有序的子数组，然后我们选两个，因为在他们内部已经有序了，所以我们比第一个数字的大小，谁大谁放第一个……就这样，两个有序数组合并成了一个大的有序数组，最终实现了排序。

先写到这里，未完待续

Mostly_Harmless

写于2023.11.20 上午十时